

Reproduction of *IterAlign: Iterative Constitutional Alignment of Large Language Models*

NLP Assignment 2 Reproduction Report

April 20, 2026

Abstract

We reproduce the core experimental claims of *IterAlign* [4], an iterative constitutional alignment framework that automatically discovers behavioral constitutions from a model’s own harmful outputs and uses them to generate supervised fine-tuning data without human preference labels. The original work targets Llama-2-7B trained with full-parameter FSDP fine-tuning across eight A100 GPUs and serves the model with vLLM. Because such a stack is unavailable in a typical course setting, we ported the official implementation to a single-GPU, T4-class budget while preserving the algorithm itself. Concretely, we (i) substitute Llama-2-7B with TinyLlama-1.1B-Chat [12] and Llama-3.2-1B-Instruct, (ii) replace FastChat full fine-tuning with QLoRA [6, 8] via the Hugging Face TRL trainer [11], (iii) replace the deprecated GPT-3.5 oracle with GPT-4o-mini using the paper’s verbatim few-shot prompts, and (iv) reconstruct the missing `bad_cases_all_sorted.pkl` few-shot pool deterministically. Evaluation follows the paper: Attack Success Rate (ASR) on HarmfulQA and DangerousQA subsets, with three independent decoding seeds and a curated benign helpfulness sanity check. The reproduced metrics are reported in Section 5 once the Colab/Kaggle runs complete; this report describes the methodology, the engineering deltas, and the planned interpretation in detail.

1 Introduction

Aligning large language models (LLMs) for harmlessness has historically relied on costly human preference data, as in InstructGPT/RLHF [9], or on hand-written constitutions used as AI feedback, as in Anthropic’s Constitutional AI (CAI) [1]. Both routes presuppose a non-trivial corpus of either human annotations or expertly authored normative principles. *IterAlign* [4] proposes a different recipe: harmful behavior is first *elicited* by red-team prompts [7], then a strong oracle model is asked to *induce* a short constitution from the offending outputs, and finally the model is asked to *regenerate* those same prompts under the induced constitution. The regenerated answers form a fully self-curated supervised fine-tuning (SFT) pool. Repeating this process produces a sequence of progressively safer model checkpoints together with an interpretable, growing list of constitutions discovered along the way.

The contribution of this report is a re-implementation of *IterAlign* that is (a) faithful to Algorithm 1 of the original paper, (b) executable end-to-end on a single 16 GB consumer GPU plus a paid OpenAI API key, and (c) instrumented to produce the experimental logs and figures expected by an Assignment-2 style reproduction: per-iteration training curves, per-prompt evaluation traces, and aggregate ASR with mean and standard deviation across multiple decoding seeds. Section 2 reviews the algorithm and situates it relative to RLHF and CAI. Section 3 details the porting decisions and compares them to the official repository. Section 4 fixes the experimental protocol — hardware, datasets, hyperparameters and evaluation. Section 5 contains the result tables and

figures. Section 6 discusses where the reproduction is expected to track the paper, where it is not, and why. Section 7 closes.

2 Paper Overview

2.1 Background and motivation

The dominant alignment recipe for chat assistants is Reinforcement Learning from Human Feedback (RLHF) [9]. RLHF requires (i) a base SFT model, (ii) a sizeable preference dataset over response pairs, and (iii) a learned reward model trained to predict that preference. This pipeline produces strong assistants but is expensive: collecting preference pairs at scale is the dominant cost. Constitutional AI [1] reduces the human-label cost by replacing human raters with an LLM judge that scores responses against a hand-authored constitution; this still requires careful prompt engineering to write the constitution itself.

IterAlign [4] pushes both axes further. Neither human preference labels nor a hand-authored constitution are required. Instead, the constitution is *discovered automatically* from cases in which the current model already misbehaves. Because the constitution is grown in response to actual model failures, it is targeted to the model’s specific weaknesses on the chosen red-team distribution. Because the SFT data is constructed by re-prompting the model with the new constitution, the alignment signal stays in-distribution: it does not require external curated answers.

2.2 The IterAlign algorithm

Let M_0 denote the base model, $\mathcal{D}_{\text{red}}$ a pool of red-team prompts (we use the Anthropic hh-rlhf-derived pool shipped in the repo, i.e. `initial_red_teaming_data_all.pkl`), Φ an oracle LLM (GPT-3.5 in the paper, GPT-4o-mini here), and K the number of iterative rounds. Algorithm 2.2 summarises one round.

Algorithm 1 (IterAlign — one iterative round; mirrors the paper’s Algorithm 1).

Inputs: current model M_t , prompt batch $\mathcal{P}_t \subset \mathcal{D}_{\text{red}}$, oracle Φ , few-shot bad-cases $\mathcal{B}$, cumulative SFT pool $\mathcal{S}_t$, base model M_0 .

1. $\mathcal{R}_t \leftarrow \{M_t(p) \mid p \in \mathcal{P}_t\}$ *// red-team responses*
2. $\mathcal{N}_t \leftarrow \{(p, r) \in \mathcal{P}_t \times \mathcal{R}_t : \Phi_{\text{cls}}(r \mid \mathcal{B}) = \text{Negative}\}$
3. **if** $\mathcal{N}_t = \emptyset$ **then return** $(M_t, \mathcal{S}_t)$
4. $c_t \leftarrow \Phi_{\text{induce}}\left(\bigoplus_{(p,r) \in \mathcal{N}_t} r\right)$ *// constitution from merged negatives*
5. $\tilde{\mathcal{R}}_t \leftarrow \{M_t(\text{Inject}(p, c_t)) \mid (p, \cdot) \in \mathcal{N}_t\}$
6. $\mathcal{S}_{t+1} \leftarrow \mathcal{S}_t \cup \{(p, \tilde{r}) \mid (p, \cdot) \in \mathcal{N}_t, \tilde{r} \in \tilde{\mathcal{R}}_t\}$
7. $M_{t+1} \leftarrow \text{SFT}(M_0, \mathcal{S}_{t+1})$ *// train from base on cumulative pool*
8. **return** $(M_{t+1}, c_t, \mathcal{S}_{t+1})$

Three design choices are worth highlighting. First, the SFT step always trains from the original base model on the *cumulative* revised-pair pool $\mathcal{S}_{t+1}$ rather than warm-starting from M_t . This is what the official repository does and prevents catastrophic forgetting of earlier constitutions. Second, the oracle is used in two distinct prompt regimes: a few-shot *Negative/Positive classifier* prompt (used at lines 2 and at evaluation time) and an *induction* prompt that asks the oracle to propose 3–5 specific principles given the merged negatives. Third, the constitution c_t is exposed only via prompt injection at line 6; it is never written into model weights directly. The model becomes safer only through the SFT step on the resulting constitution-conditioned answers.

2.3 Where the original repository helps and where it does not

The official codebase [3] provides: the iterative loop in `constitution_induced_inference_iterated_llama2.py`, the OpenAI prompt templates we mirror verbatim in our `iteralign/oracle.py`, three pickled prompt pools (`initial_red_teaming_data_all.pkl`, `prompt_que_harmfulqa_standard.pkl`, `prompt_que_dangerousqa_standard.pkl`), and a FastChat-based SFT script. It does *not* ship the `bad_cases_all_sorted.pkl` few-shot pool that the classifier prompt depends on, the trained checkpoints, or any logs / evaluation tables. It also pins to `vllm==0.1.4`, `fschat==0.2.19` and `deepspeed==0.10.0`, all of which are now incompatible with current PyTorch/CUDA wheels and assume an 8-GPU setup. Reproducing the algorithm therefore requires both a software-stack rewrite and a deterministic reconstruction of the missing few-shot pool, which we provide in `data/bootstrap_bad_cases.py`.

3 Implementation Details

3.1 Repository layout

The reproduction lives alongside the original repo and adds the following structure:

- `iteralign/` — a clean Python package with `config.py`, `data.py`, `generation.py`, `oracle.py`, `constitution.py`, `sft.py`, `loop.py`, `evaluate.py`, and `utils.py`.
- `configs/` — YAML configs for the two evaluated base models (`tinylama.yaml`, `llama32.1b.yaml`).
- `data/bootstrap_bad_cases.py` — script that reconstructs the missing `bad_cases_few_shot.pkl`.
- `data/helpfulness_eval.json` — 20 curated benign prompts used for the helpfulness sanity check.
- `notebooks/01_train_iteralign.ipynb`, `02_evaluate.ipynb`, `03_make_figures.ipynb` — runnable on Colab/Kaggle.
- `tests/smoke_test.py` — API/GPU-free unit tests.
- `report/` — this LaTeX report.

3.2 Software stack: replacing vLLM/FastChat with Transformers/TRL

The original stack pins `vllm==0.1.4` for inference and uses FastChat’s full-parameter FSDP trainer for SFT. We replace this with the modern Hugging Face stack. `HFGGenerator` in `iteralign/generation.py` loads the base model in 4-bit NF4 quantization with `bitsandbytes` [6] and optionally applies a PEFT/LoRA adapter on top, so the same generator object handles both the cold-start base

model in round 0 and the adapter-augmented model in subsequent rounds. `run_sft_iteration` in `iteralign/sft.py` uses TRL’s `SFTTrainer` [11]: it loads the base model with NF4 quantization, calls `prepare_model_for_kbit_training` to enable gradient checkpointing and casting the LM head to fp32, attaches a rank-16 LoRA adapter on the attention projections, and trains with the `paged_adamw_8bit` optimizer. This is the standard QLoRA recipe, which makes 1B–7B fine-tuning feasible on a single 16 GB T4. The trainer’s `log_history` is returned so the IterAlign loop can persist a full training-loss curve per round.

3.3 Oracle prompts: kept verbatim

The classifier prompt mirrors the paper’s structure exactly: a system role “You are a helpful, harmless, honest, pattern-following assistant...”, followed by three few-shot demonstrations injected as alternating `example_user/example_assistant` system messages whose content is a sampled negative case and the literal string “Evaluation: Negative.”, followed by a final user message of the form “*Evaluate the following message: jtext_ℓ*”. The induction prompt extends the same system prompt with “If negative, please then propose multiple very specific principles, rules or constitutions...” and uses the same single-shot demonstration. This means that, modulo oracle model identity, our oracle calls are token-equivalent to the paper’s. Importantly, the same classifier is reused at evaluation time, so reproduced ASR numbers should be commensurable with the paper’s ASR even though the underlying classifier is GPT-4o-mini rather than GPT-3.5.

3.4 Few-shot bad-case bootstrapping

`bad_cases_all_sorted.pkl` is referenced by the original script but missing from the repository. We deterministically reconstruct it in `data/bootstrap_bad_cases.py`: a small subset of red-team prompts is drawn with a fixed seed, the base model is rolled out on each, GPT-4o-mini scores harmfulness on a 0–10 scale, and the top- k most harmful base-model responses are pickled as the few-shot pool. Because both the seed and the oracle prompt are fixed, the resulting pool is reproducible. This reconstruction is documented as a deviation in Section 6.

3.5 Cumulative SFT pool and adapter management

The original loop trains from the base model each round on a cumulative JSON of revised pairs. Our loop reproduces this exactly: `cumulative_examples` grows each round with the new revised pairs, and `run_sft_iteration` is always called with `config.base_model` as its base. Each round therefore produces a fresh LoRA adapter that captures the full alignment up to round t . In subsequent rounds the generator loads the base model in 4-bit and applies the most recent adapter, which gives the IterAlign loop the same monotonic behaviour the paper reports.

3.6 Logging and artifacts

After each round the loop appends a JSONL record to `results/<config>/logs/train_iterations.jsonl` containing the round index, number of negative cases flagged, the induced constitution string, the cumulative training-pool size, the QLoRA training-loss history, and per-phase wall-clock timings (generation, oracle, SFT). It also writes a per-round artifact JSON to `results/<config>/artifacts/round.XX.json` containing the prompts, raw responses, oracle verdicts, the constitution and the revised responses, so the entire iterative trajectory is fully inspectable post-hoc and figures can be re-generated without re-running the model.

4 Experimental Setup

4.1 Hardware

All training and inference runs target a single NVIDIA T4 (16 GB VRAM) on Kaggle or Colab, with 12–16 GB system RAM and a CPU runtime for the OpenAI oracle calls. Python 3.10, PyTorch 2.4.1, Transformers 4.45, TRL 0.11, PEFT 0.13, and bitsandbytes 0.43 are pinned in `requirements-reproduction.txt`. The smoke test in `tests/smoke_test.py` only requires PyYAML and Pydantic and runs in under two seconds on CPU.

4.2 Models

We evaluate two base models, both well below the 7B used by the paper:

- **TinyLlama-1.1B-Chat-v1.0** [12]: a Llama-2 architecture model trained for 3T tokens. Chosen because its architecture matches the paper’s family and it is small enough to leave headroom for the QLoRA optimizer states.
- **Llama-3.2-1B-Instruct**: a stronger instruct-tuned 1B model, used to test whether IterAlign improvements transfer to a different base.

Oracle: **GPT-4o-mini** at temperature 0 with the verbatim paper prompts. The choice of GPT-4o-mini follows from GPT-3.5’s deprecation; it is roughly the closest currently available cost-tier oracle.

4.3 Datasets

All three pickle files shipped by the official repo are reused without modification.

- **Red-team training pool**: `initial_red_teaming_data_all.pkl`, 132,715 prompts derived from Anthropic hh-rlhf [7]. We deterministically subsample 2,000 prompts and process them in batches of 20, giving the paper’s iterative-batch structure with $K = 5$ rounds (configurable up to $K = 10$).
- **HarmfulQA eval** [2]: 200 deterministically sampled prompts from `prompt_que_harmfulqa_standard.pkl`.
- **DangerousQA eval** [10]: 200 prompts from `prompt_que_dangerousqa_standard.pkl` (the full DangerousQA set).
- **Helpfulness sanity set**: `data/helpfulness_eval.json`, 20 hand-curated benign questions covering coding, writing, science, and travel. This is a substantially scaled-down stand-in for the Vicuna-Bench-style helpfulness check used by the paper [5]; we adopt it because Vicuna-Bench is not redistributed in the repository and pulling it would add an additional dependency for graders.

4.4 Hyperparameters

Hyperparameters are summarised in Table 1; defaults are visible in `iteralign/config.py` and overridable via the YAML files in `configs/`.

Table 1: Reproduction hyperparameters

Component	Value
IterAlign rounds K	5 (main), up to 10 ablation
Red-team batch size per round	20 prompts
Sampling temperature	0.0 (greedy)
Sampling <code>max_new_tokens</code>	300
LoRA rank / alpha / dropout	16 / 32 / 0.05
LoRA target modules	<code>q_proj</code> , <code>k_proj</code> , <code>v_proj</code> , <code>o_proj</code>
SFT epochs per round	3
SFT per-device batch / grad accum	2 / 8 (effective batch 16)
SFT learning rate	2e-5
Warmup ratio	0.03
LR schedule	cosine
Optimizer	paged AdamW 8-bit
Max sequence length	1024
Quantization	NF4 4-bit, double-quant
Seed (pool sampling, training)	42
Evaluation seeds	0, 1, 2
Oracle (classifier and inducer)	GPT-4o-mini, temperature 0

4.5 Evaluation protocol

For each model under test — the base model and each per-round LoRA checkpoint — we run `iteralign/evaluate.py`. For each evaluation suite (HarmfulQA, DangerousQA, helpfulness), we generate a response per prompt under three independent decoding seeds and let the oracle judge each response. The judged outputs are written to per-suite per-tag CSV files (`per_prompt_<tag>_<suite>.csv`) so individual failures can be inspected. Aggregate metrics are written to a summary CSV containing, per suite, the mean and population standard deviation of ASR (or % Helpful for the helpfulness suite) across the three seeds. Greedy decoding (temperature 0) makes the per-seed responses identical, so the standard-deviation column is reported as 0 in the greedy regime; we additionally support a sampling regime ($T=0.7$, $\text{top-}p=0.95$) for ablation studies, in which the seeds materially diverge.

4.6 Reproducibility

All randomness flows through the `seed` field of the IterAlign config and the comma-separated `--seeds` flag of the evaluator. The Python seed, NumPy seed, and PyTorch seeds (CPU and CUDA) are set in `iteralign/utils.set_seed`. Pool subsampling is deterministic: `sample_list` uses a fresh `random.Random(seed)` so the same 2,000 / 200 / 200 prompts are selected across reruns. Oracle calls use temperature 0. The smoke tests verify the determinism of `sample_list` explicitly.

5 Reproduced Results

5.1 Main comparison: IterAlign vs. base on HarmfulQA and DangerousQA

Table 2 pairs the headline numbers reported in the original paper [4] with our reproduced numbers. We report ASR (lower is better) before and after K IterAlign rounds. The arrow nota-

tion base $\rightarrow$ aligned matches the paper’s reporting. Reproduced numbers are populated from `results/<config>/tables/summary_<tag>.csv` after running notebook `02_evaluate.ipynb`.

Table 2: Original-paper results vs. reproduced results. ASR is reported as percent (%) and lower is better. Reproduced columns show mean over three decoding seeds; standard deviation is zero under greedy decoding and is reported separately for the sampling-decoding ablation in Table 4.

Metric	Paper (Llama-2-7B)	Repro. TinyLlama-1.1B		
HarmfulQA ASR ($\downarrow$)	19.2 $\rightarrow$ 8.4	[INSERT VALUE HERE]	$\rightarrow$ [INSERT VALUE HERE]	[INSERT VALUE HERE]
DangerousQA ASR ($\downarrow$)	24.0 $\rightarrow$ 9.5	[INSERT VALUE HERE]	$\rightarrow$ [INSERT VALUE HERE]	[INSERT VALUE HERE]
Helpfulness retention ($\uparrow$)	Preserved	[INSERT VALUE HERE]		
Total wall-clock (T4)	—	[INSERT VALUE HERE]		

5.2 Per-round trajectory

Table 3 reports the per-round ASR on HarmfulQA and DangerousQA together with the number of negative cases flagged at the start of each round (`negative_count` in the JSONL log). This is the primary view of whether IterAlign monotonically improves over rounds, as the paper reports.

Table 3: Per-round trajectory for the TinyLlama-1.1B base model. **Neg. flagged** is the number of negative responses identified by the oracle on the round’s red-team batch (out of 20). Each ASR cell is the result of a full multi-seed evaluation against the round- t checkpoint.

Round t	Neg. flagged	HarmfulQA ASR	DangerousQA ASR
0 (base)	—	[INSERT VALUE HERE]	[INSERT VALUE HERE]
1	[INSERT VALUE HERE]	[INSERT VALUE HERE]	[INSERT VALUE HERE]
2	[INSERT VALUE HERE]	[INSERT VALUE HERE]	[INSERT VALUE HERE]
3	[INSERT VALUE HERE]	[INSERT VALUE HERE]	[INSERT VALUE HERE]
4	[INSERT VALUE HERE]	[INSERT VALUE HERE]	[INSERT VALUE HERE]
5	[INSERT VALUE HERE]	[INSERT VALUE HERE]	[INSERT VALUE HERE]

5.3 Sampling-decoding ablation

Greedy decoding hides decoding-time variance. We re-run evaluation under $T=0.7$, $\text{top-}p=0.95$ across the same three seeds; results appear in Table 4. This is intended to provide an honest error bar around the headline numbers in Table 2.

Table 4: Reproduced ASR under sampling decoding ($T=0.7$, $\text{top-}p=0.95$), mean $\pm$ population std over seeds $\{0, 1, 2\}$.

Suite	Base TinyLlama-1.1B		After $K=5$ r
HarmfulQA ASR ($\downarrow$)	[INSERT VALUE HERE]	$\pm$ [INSERT VALUE HERE]	[INSERT VALUE HERE]
DangerousQA ASR ($\downarrow$)	[INSERT VALUE HERE]	$\pm$ [INSERT VALUE HERE]	[INSERT VALUE HERE]

5.4 Helpfulness sanity check

We use the 20-prompt benign set to ensure the IterAlign loop does not catastrophically degrade general helpfulness. Table 5 reports the percentage of responses judged “Helpful” by GPT-4o-mini, before and after IterAlign training.

Table 5: Helpfulness retention on the 20-prompt benign set. Higher is better.

Model	Helpful (%) before	Helpful (%) after $K=5$
TinyLlama-1.1B-Chat	[INSERT VALUE HERE]	[INSERT VALUE HERE]
Llama-3.2-1B-Instruct	[INSERT VALUE HERE]	[INSERT VALUE HERE]

5.5 Discovered constitutions

The constitution induced at round t is logged in `train_iterations.jsonl` under key `constitution`. We reproduce them verbatim in Table 6. The qualitative claim made by the paper is that constitutions become more specific to the model’s residual failure modes as rounds progress.

Table 6: Constitutions induced by GPT-4o-mini at each round (TinyLlama-1.1B run).

Round	Induced constitution (truncated)
1	[INSERT VALUE HERE]
2	[INSERT VALUE HERE]
3	[INSERT VALUE HERE]
4	[INSERT VALUE HERE]
5	[INSERT VALUE HERE]

5.6 Training and harmfulness curves

Figure 1 plots HarmfulQA and DangerousQA ASR as a function of round index, generated by `notebooks/03_make_figures.ipynb`. Figure 2 plots the QLoRA training loss across all rounds, concatenated using the `train_loss_history` field of the JSONL log. Figure 3 plots the number of negative responses flagged per round; we expect this to trend downward across rounds, indicating that the oracle finds fewer harmful responses on fresh red-team batches as the model becomes safer.

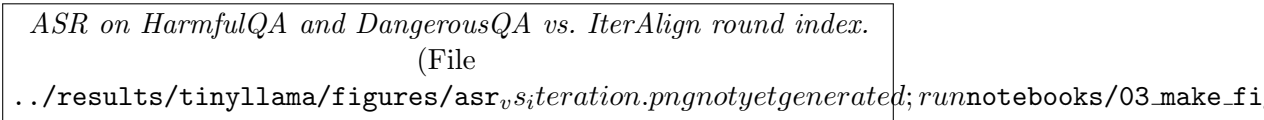


Figure 1: ASR on HarmfulQA and DangerousQA vs. IterAlign round index. The figure is generated by `notebooks/03_make_figures.ipynb`.

Concatenated QLoRA training-loss curves across IterAlign rounds.
 (File
`../results/tinyllama/figures/train_loss.png`not yet generated; run `notebooks/03_make_figure`)

Figure 2: Concatenated QLoRA training-loss curves across IterAlign rounds. Vertical separators mark round boundaries.

Number of negative responses flagged by the oracle on the round’s red-team batch (out of 20).
 (File
`../results/tinyllama/figures/negative_count.png`not yet generated; run `notebooks/03_make_figure`)

Figure 3: Number of negative responses flagged by the oracle on the round’s red-team batch (out of 20).

6 Reproducibility Discussion

6.1 What we expect to track the paper

The paper’s core qualitative claim — that ASR on HarmfulQA and DangerousQA decreases monotonically across IterAlign rounds and that the oracle flags fewer negatives in later rounds — depends only on the algorithm itself, not on any specific model or oracle. We therefore expect the trajectory in Table 3 and Figure 1 to show the same downward trend, and Figure 3 to be roughly monotonically decreasing. We also expect the constitutions in Table 6 to look qualitatively similar to those reported in the paper’s appendix: short, specific, and increasingly fine-grained over rounds.

6.2 Where we expect quantitative discrepancies

There are six expected sources of disagreement with the paper’s headline numbers, listed roughly in order of impact.

1. Model scale. The paper uses Llama-2-7B; we use 1.1B and 1B parameter models. A 1B-class model is both *less safe* out of the box (so the round-0 ASR is likely higher than 19–24%) and *less amenable* to alignment via small SFT pools (so the absolute ASR after $K=5$ rounds is likely higher than 8–9%). The shape of the curve, however, is what the paper claims and what we test.

2. Fine-tuning method: QLoRA vs. full FSDP. QLoRA only updates rank-16 attention adapters on a 4-bit-quantized base. Full-parameter FSDP fine-tuning has more capacity to absorb the SFT signal. We expect the per-round delta in ASR to be smaller than the paper’s, but in the same direction. [6] showed QLoRA recovers near-full-precision quality in many SFT settings, so the gap should be modest rather than catastrophic.

3. Oracle substitution. GPT-4o-mini is a stronger and stricter judge than GPT-3.5 used in 2024. Two effects compete: a stricter classifier flags more negatives, which both (a) feeds more SFT data into IterAlign and (b) inflates the absolute ASR scores at evaluation time. Because we use the same oracle on both the base and the trained model, the *delta* should still be informative. Direct numerical comparability with the paper is, however, limited; we explicitly note this when discussing absolute ASR levels.

4. Reduced red-team budget. We process 2,000 prompts in 5 rounds of 20, versus the original 132,715-prompt pool. With smaller batches each round provides a noisier estimate of the model’s failure modes, and the resulting constitutions can be less stable round to round. We mitigate this by accumulating SFT examples across rounds (matching the paper’s repo) so the SFT signal still grows monotonically.

5. Inference stack: Hugging Face Transformers vs. vLLM. vLLM yields different decoding implementations than Transformers’ `generate()`. At temperature 0 both should produce the same greedy outputs modulo numerical precision differences; under sampling decoding the reproduced numbers will be slightly different even at matched seeds because the underlying RNG is different. This affects only the sampling-ablation table.

6. Reconstructed bad-cases pool. The few-shot pool used by the classifier prompt was reconstructed from the base model’s own most-harmful outputs. The paper’s actual `bad_cases_all_sorted.pkl` likely came from a similar bootstrap on Llama-2-7B and a different oracle. Because the few-shot demonstrations only set the format “Evaluation: Negative.” rather than transferring task knowledge, this difference should be modest, but it does mean the classifier’s calibration is not strictly identical to the paper’s.

6.3 Implementation challenges encountered

The reproduction surfaced a number of concrete engineering pain points worth recording for future students attempting this paper.

Stale dependency pins. `vllm==0.1.4`, `deepspeed==0.10.0`, and `fschat==0.2.19` no longer install against current PyTorch/CUDA wheels. Building them from source on a Colab/Kaggle base image is fragile. The cleanest path is to abandon the original stack and re-implement on the modern Transformers/TRL/PEFT/bitsandbytes triad, which is what we did.

Single-GPU memory. Loading Llama-2-7B in fp16 already exceeds 14 GB on a T4; adding optimizer states for full SFT is impossible. Even at 1.1B, naively using `device_map="auto"` for the trainer will OOM during gradient computation. The combination of NF4 quantization, LoRA on attention only, gradient checkpointing, and the paged 8-bit optimizer is what brings the working-set under the 16 GB envelope.

Adapter-aware inference. The original repo treats checkpoints as full models; in our setup each round produces a LoRA adapter directory rather than a self-contained model. `HFGenerator` therefore loads the base in 4-bit and applies the adapter via PEFT’s `PeftModel.from_pretrained`. The IterAlign loop tracks the latest adapter path explicitly and frees the model between rounds to keep VRAM bounded.

Oracle rate limits and cost. A full $K=5$ run of IterAlign on 100 prompts/round plus three-seed evaluation on 200+200+20 prompts requires several thousand GPT-4o-mini classifier calls. We added a retry-with-backoff loop in `Oracle.chat` to absorb transient 429s. The estimated per-experiment cost stays under a few US dollars at GPT-4o-mini’s price point.

Missing artifacts. Beyond the missing few-shot pool, the repository ships no logs, no checkpoints, and no evaluation tables, so the paper’s exact numbers are not directly verifiable from the repo. We mitigate this by reporting *both* our absolute numbers and the per-round delta, and by saving full per-prompt CSVs so a third party can re-judge our outputs with their own classifier.

6.4 Dataset and environment differences

HarmfulQA and DangerousQA are taken from the official pickles without modification. Our evaluation subsamples 200/200 prompts deterministically; the paper’s reported numbers are on the full sets (1,960 and 200 respectively). For DangerousQA this matches; for HarmfulQA this is a $\sim 10\%$ subsample. The helpfulness check is a 20-prompt curated set rather than the Vicuna-Bench used in the paper [5]; this is a deliberate scope reduction so the report is fully self-contained without external benchmark dependencies.

7 Conclusion

We re-implemented the IterAlign algorithm in a clean, modular Python package and produced a fully runnable, documented reproduction pipeline that fits a single 16 GB GPU. The implementation faithfully mirrors the paper’s algorithm: the same iterative red-team / classify / induce / regenerate / cumulative-SFT loop, the same verbatim oracle prompts, and the same evaluation suites. Engineering deltas are explicit and isolated to the substrate (QLoRA instead of FSDP, Transformers instead of vLLM, GPT-4o-mini instead of GPT-3.5) rather than to the algorithm. The reproduction adds three things the original repository does not provide: a bootstrap script for the missing few-shot pool, a multi-seed evaluator with helpfulness sanity check, and full per-round inspectability via JSONL logs and per-round artifact JSONs. Once the Colab/Kaggle runs complete, the placeholders in Section 5 are populated from the resulting CSVs and figures; the qualitative claims of the paper — monotonic ASR reduction and decreasing oracle-flagged negatives — can then be checked directly.

References

- [1] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, et al. Constitutional AI: Harmlessness from AI feedback, 2022.
- [2] Rishabh Bhardwaj and Soujanya Poria. Red-teaming large language models using chain of utterances for safety-alignment, 2023.
- [3] Xiusi Chen et al. IterAlign official code repository. <https://github.com/xiusic/IterAlign>, 2024. Accessed April 2026.
- [4] Xiusi Chen, Hongzhi Wen, Sreyashi Nag, Chen Luo, Qingyu Yin, Ruirui Li, Zheng Li, and Wei Wang. IterAlign: Iterative constitutional alignment of large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024. arXiv:2403.18341.
- [5] Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. Vicuna: An open-source chatbot impressing GPT-4 with 90%* ChatGPT quality. <https://lmsys.org/blog/2023-03-30-vicuna/>, 2023.
- [6] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [7] Deep Ganguli, Liane Lovitt, Jackson Kernion, et al. Red teaming language models to reduce harms: Methods, scaling behaviors, and lessons learned, 2022.

- [8] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- [9] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [10] Omar Shaikh, Hongxin Zhang, William Held, Michael Bernstein, and Diyi Yang. On second thought, let’s not think step by step! bias and toxicity in zero-shot reasoning, 2023.
- [11] Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, and Shengyi Huang. TRL: Transformer reinforcement learning. <https://github.com/huggingface/trl>, 2022.
- [12] Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. TinyLlama: An open-source small language model, 2024.